Lab 22 – Advanced Ethical Considerations: Addressing Complex Scenarios and Limitations

NAME : M.Dinesh

ROLL-NO : 2503A52374

Batch-13

Task 1 – Bias Detection in AI Suggestions

Aim

To identify possible biases in AI-generated coding suggestions by providing the same coding task with different prompt descriptions and comparing the results.

Input (Prompts Given to AI)

Prompt 1:
"Write a Python program to select the best candidate for a job based on marks and experience."

Prompt 2:
"Write a Python program to select a job candidate fairly based on skills, qualifications and experience without considering gender, age or background."

Code:

```
employees = [
    {"name": "Ali", "experience": 5, "skills": 80, "qualification": 85},
    {"name": "Sara", "experience": 2, "skills": 95, "qualification": 90},
    {"name": "John", "experience": 4, "skills": 75, "qualification": 80}
]

# Selection based on experience
best_experience = max(employees, key=lambda x: x["experience"])
print("Prompt 1 - Selected Employee:", best_experience["name"])

# Selection based on skills and qualifications
best_skills = max(
    employees,
    key=lambda x: (x["skills"] + x["qualification"]) / 2
)
print("Prompt 2 - Selected Employee:", best_skills["name"])
```

Output:



Discussion:

- Hidden assumptions or biases: Experience-based selection may favour experienced employees and disadvantage freshers.
- Impact on fairness: Biased AI code can reject qualified candidates unfairly.
- Impact on accessibility: Systems that ignore accessibility needs may disadvantage people with disabilities.

- Solution: Use fair evaluation criteria, test AI-generated code and involve humans in important decisions

Task 2 – Privacy and Sensitive Data Handling

Aim:

To understand how AI-generated code handles personal information and learn methods to protect sensitive user data through anonymization and responsible data processing.

Input (Prompt Given to AI):

"Write a Python program to process user personal data such as names and phone numbers. Apply data anonymization by hiding personal details and displaying only partially masked information."

Code:

```
users = [
    {"name": "Ali Khan", "phone": "9876543210"},
    {"name": "Sara Begum", "phone": "9123456780"},
    {"name": "John David", "phone": "9988776655"}
]


def anonymize_name(name):
    return name[0] + "*" * (len(name) - 1)


def anonymize_phone(phone):
    return "*" * 6 + phone[-4:]
```

```
print("Anonymized User Data:")


for user in users:
    print("Name:", anonymize_name(user["name"]))
    print("Phone:", anonymize_phone(user["phone"]))
```

Output:



Discussion:

- Data Anonymization: Hide or replace personal details such as names, phone numbers and email addresses. Use masking, encryption or pseudonymization where appropriate.
- Ethical Issues: Exposing personal information can cause identity theft, fraud and loss of privacy.
- Legal Issues: Mishandling personal data may violate privacy laws such as India's Digital Personal Data Protection Act, 2023.
- Developer Responsibility: Developers must protect user information, obtain appropriate consent, restrict data access, avoid unnecessary data collection and test AI-generated code before deployment.

Task 3 – AI-Generated Security Risks

Aim:

To generate login authentication code using AI, identify possible security vulnerabilities and understand ethical guidelines for secure AI-generated code.

Input (Prompt Given to AI):

"Write a simple Python login authentication program that accepts a username and password and verifies the user's credentials."

Code:

```
# AI-generated basic login program


users = {
    "admin": "admin123",
    "student": "123456"
}


username = input("Enter username: ")
password = input("Enter password: ")


if username in users and users[username] == password:
    print("Login Successful")
else:
    print("Invalid username or password")
```

Output:



Security Risks:

1. Weak Password Storage: Passwords are stored as plain text instead of being securely hashed.

2. No Encryption: Sensitive information may be exposed during transmission.
3. Weak Passwords: Simple passwords can be guessed easily.
4. No Login Protection: Unlimited login attempts can allow brute-force attacks.
5. Information Exposure: User credentials may be exposed if the database is accessed by unauthorized people.

Ethical Guidelines for Safe AI Code Usage

1. Never store passwords in plain text.
2. Use secure password hashing algorithms such as Argon2 or bcrypt.
3. Use HTTPS to protect data during transmission.
4. Implement login attempt limits and account protection.
5. Review and test AI-generated code before using it.
6. Never trust AI-generated security code without human verification.

Task 4 – Accountability in Human–AI Collaboration

Aim:

To understand accountability and human responsibility when AI-generated code is used in critical applications such as hospital patient monitoring systems.

Input (Prompt Given to AI):

"Generate a simple Python patient monitoring program that checks a patient's heart rate and displays an alert when the heart rate is abnormal."

Code:

```
heart_rate = int(input("Enter patient's heart rate: "))


if heart_rate < 60:
    print("Warning: Low heart rate")
elif heart_rate > 100:
    print("Warning: High heart rate")
else:
    print("Heart rate is normal")
```

Output:



Reflection Report:

1. Is the AI tool responsible?

AI generates code based on the given instructions. It does not have human judgment or legal accountability. Therefore, the AI tool itself cannot take full responsibility for the harm caused.

2. Is the human developer responsible?

Yes. The developer must review, test and validate AI-generated code before using it in a real hospital system.

3. Is the organization responsible?

Yes. The hospital or organization must ensure proper testing, approval, maintenance and monitoring of the system.